

ROUND: keep the promise specific

A small check can earn a useful amount of trust

01 / THE SAS EXAMPLE

```
data test;
  input value;
  rounded = round(value);
  datalines;
2.3
2.7
-1.4
-1.6
3.0
;
run;

proc print data=test noobs;
run;
```

ATTACH SOURCE OR PASTE IT AFTER THE PROMPT. START FRESH EACH TIME.

WHAT YOU WILL PRACTICE

Match a simple conversion to an explicit scope and a compact result check.

Round five positive and negative numbers to the nearest integer. None is a halfway value. This exercise establishes agreement only for these supplied values.

EXPECTED RESULT

Row	value	rounded
1	2.3	2
2	2.7	3
3	-1.4	-1
4	-1.6	-2
5	3.0	3

Compare numeric values. A display of 2.0 equals 2 here. Halfway cases and other rounding units are outside this 101 example.

Expected values checked in Python; SAS execution pending.

Prompt A: leave choices open

High degrees of freedom / Notice the decisions the model makes for you.

COPY THIS PROMPT + THE SAS EXAMPLE

Convert this SAS code to Python.

Show code in this response. Do not create or run files, or read existing Python files.

LOOK AT THE RESPONSE

Which library and method did it choose? Did it preserve the expected behavior? Compare the result with page 1 before accepting it.

TRY IT / THREE FRESH ATTEMPTS

Run A three times with the same SAS example and setup. Save each first response before asking follow-up questions. Record agreement as well as differences.

Bedrock A/B trials for this example are pending.

WHY CHOICES REMAIN OPEN

1 A recognizable task

The code supplies five inputs and integer rounding. A short request may already produce a correct conversion.

2 Several valid approaches

Built-in Python, NumPy, and pandas can agree here. Different library choices are not automatically numerical errors.

3 The tested scope matters

Success on these values supports this example. It does not cover every SAS rounding case.

ANTI-PATTERN TO AVOID

Anti-pattern: treating five matching answers as proof that all ROUND cases are interchangeable. Keep the claim tied to these values. Add checks when the real task introduces new cases.

Prompt B: make the choices explicit

Low degrees of freedom / State the behavior, then ask for a check.

COPY THIS PROMPT + THE SAME SAS EXAMPLE

Convert this SAS code to Python using pandas.

- Build a DataFrame with value in the original row order: [2.3, 2.7, -1.4, -1.6, 3.0].
- Create rounded using the value Series .round(0) method.
- Print value and rounded. Numeric 2 and 2.0 are equivalent here.
- Include an assertion that rounded values are [2, 3, -1, -2, 3].
- Briefly explain what the check covers: these five inputs, nearest-integer rounding, including the negative inputs.
- State that halfway values and other rounding units have not been tested. Keep those cases outside this exercise.

Show code in this response. Do not create or run files, or read existing Python files.

WHY THESE DETAILS HELP

1 Choose one route

Naming pandas and its rounding method reduces implementation choices while keeping the task simple.

2 Define agreement

The expected numeric list is the finish line. A display difference such as 2 versus 2.0 is harmless here.

3 Bound the conclusion

State what was checked so a useful local result does not become an unsupported general claim.

CHECK IT YOURSELF

Run the code. Compare [2, 3, -1, -2, 3] in order and confirm inputs are unchanged. Record a pass for this dataset; halfway cases and other rounding units remain untested.

Earn trust for a stated scope, then extend that scope with new examples and checks.

SAS REFERENCE